# Let's Encrypt Phase-5 RENEWAL + ROTATION — Deep Research (Caddy in-process renewal vs Pebble ARI, zero-downtime rotation proof)

**Revision:** 1 **Last modified:** 2026-07-01T09:45:35Z **Authority:** Helix Constitution §11.4.150 (deep multi-angle research before workstream commitment) + §11.4.99 (latest-source cross-reference) + §11.4.6 (no-guessing — every claim cited or marked UNCONFIRMED) + §11.4.107/§11.4.115 (rock-solid rotation proof). **Scope:** design the hermetic Phase-5 RENEWAL + ROTATION test against the already-built hermetic stack — Caddy image `localhost/helix_proxy/caddy-challtestsrv:2.8.4`, Pebble `ghcr.io/letsencrypt/pebble:v2.6.0`, `pebble-challtestsrv`, per `deploy/letsencrypt/compose.hermetic.yml` + `deploy/letsencrypt/Caddyfile`. **Consumes:** `tests/letsencrypt/cert_analyzer.sh` (`cert_chain_roots_in` / `cert_not_expired` / `cert_san_matches` / `cert_days_remaining` / `cert_renewal_due`, with the `CERT_ANALYZER_NOW_EPOCH` now-seam). **Prior research this REFINES:** `docs/research/letsencrypt_hermetic_20260701/ANALYSIS.md` Q4 + gotcha #10. That doc recommended `renewal_window_ratio 1` + reload but marked "whether this Pebble build serves ARI" and "whether ARI overrides the ratio" as **UNCONFIRMED**. **This doc resolves both from source at the EXACT deployed versions** and vindicates the lever (see Q1).

> **Method note (§11.4.6 / §11.4.99):** the load-bearing facts below are read directly from the source of the **deployed** versions — Pebble `v2.6.0` (`wfe/wfe.go`, `core/types.go`, `ca/ca.go`, `test/config/pebble-config.json` at tag `v2.6.0`) and the certmagic/acmez that Caddy `2.8.4` bundles (`go.mod@v2.8.4` -> `certmagic v0.21.3` + `acmez/v2 v2.0.1`, read `certificates.go`/`config.go`/`maintain.go` at `certmagic@v0.21.3`) — fetched via `gh api` 2026-07-01. Version-mapped facts (release dates, PR->tag) come from the GitHub Releases/PR API. Where a runtime latency or an unread code path cannot be proven from source, it is marked **UNCONFIRMED** with the exact runtime method to obtain it — never guessed.

---

# Executive answer (the three asks)

1. **Recommended force-renewal mechanism: `renewal_window_ratio 1` loaded via the admin API `POST /load` graceful reload** (the `CADDY_RENEWAL_RATIO` lever already stubbed in `deploy/letsencrypt/Caddyfile`). **Winning rationale (source-proven for Caddy 2.8.4 / certmagic v0.21.3):** on config load Caddy re-manages the cert and runs `certNeedsRenewal`; with ratio `1`, `currentlyInRenewalWindow(notBefore, notAfter, 1.0)` computes `renewalWindowStart = notAfter - lifetime = notBefore`, so it returns `time.Now().After(notBefore)` = **TRUE for any already-issued cert** -> immediate in-process renewal + hot-swap. This ratio check runs **unconditionally after** the ARI block (which can only trigger renewal *earlier*, never *suppress* the ratio check), so it is **not defeated by Pebble's ARI** — resolving the prior doc's gotcha #10. It needs **no Pebble version bump, no revocation, no storage surgery**, and is the only lever that is both deterministic AND zero-downtime on the deployed stack.
2. **Concrete zero-downtime observable:** the served leaf's **serial number** and **notBefore** both CHANGE across the swap (`openssl s_client -connect ... | openssl x509 -noout -serial -startdate`) WHILE a tight continuous TLS-fetch probe records **0 failed requests / 0 dropped connections** — corroborated by the certmagic log line `certificate renewed successfully` (exact string, verified in source). A changed serial + advanced notBefore proves a real re-issue (not an in-memory cache reload).
3. **File written:** `docs/research/letsencrypt_renewal_20260701/ANALYSIS.md` (+ `.html` + `.pdf`). **Cited source count:** 24 distinct URLs (see footer) plus 7 pinned source-file reads at exact version tags.

---

# Q1 — Forcing an immediate, deterministic renewal: four levers compared

## The core mechanism (source-confirmed, Caddy 2.8.4 = certmagic v0.21.3)

`certmagic@v0.21.3/certificates.go:certNeedsRenewal(leaf, ari, ...)` is the single decision function. Its structure (verbatim shape):

```
if !cfg.DisableARI {
    selectedTime := ari.SelectedTime           // (or a random
```

```
pick inside SuggestedWindow)
    if !selectedTime.IsZero() {
        cutoff := ari.SelectedTime - RenewCheckInterval
        if time.Now().After(cutoff) { return
true }              // ARI window reached -> renew EARLY
        if currentlyInRenewalWindow(notBefore, exp, 1.0/20.0)
{ return true }  // emergency (past 95% life)
    }
}
// runs UNCONDITIONALLY (the ARI block only ever early-returns
true; it never returns false)
if currentlyInRenewalWindow(notBefore, exp,
cfg.RenewalWindowRatio) {           // the ratio check
    return true
}
if currentlyInRenewalWindow(notBefore, exp, 1.0/50.0) ||
   time.Until(exp) < RenewCheckInterval*5 { return
true }        // imminent-expiry safety net
return false
```

and `certmagic@v0.21.3/certificates.go:currentlyInRenewalWindow`:

```
lifetime           = notAfter - notBefore
renewalWindow      = lifetime * renewalWindowRatio
renewalWindowStart = notAfter - renewalWindow
return time.Now().After(renewalWindowStart)
```

**Key deduction (the whole basis of the recommendation):** with `renewalWindowRatio = 1.0`, `renewalWindowStart = notAfter - lifetime = notBefore`, so the ratio check returns `time.Now().After(notBefore)` **= TRUE for any issued cert**, immediately. And because this check sits **after** the `if ! cfg.DisableARI { ... }` block — not inside an `else` — **ARI cannot suppress it**. certmagic's ARI/ratio relationship is therefore an **OR** (renew if ARI says so **OR** the ratio window is reached **OR** expiry is imminent), NOT "ARI wins over ratio." The widely-repeated "ARI takes precedence over `renewal_window_ratio`" statement (Caddy docs, community threads) is true only in the narrow sense that **ARI can trigger a renewal *earlier* than the ratio would** — it does not, and cannot, *prevent* a ratio-triggered renewal. **This is why `renewal_window_ratio 1` deterministically forces renewal even though Pebble v2.6.0 serves an ARI window far in the future (Q3).**

**Renewal trigger timing (source-confirmed):** Caddy runs its maintenance loop on a `time.NewTicker(RenewCheckInterval)`; `certmagic v0.21.3` default `RenewCheckInterval = 10 * time.Minute` — but the test must NOT wait for the ticker. On config load Caddy re-manages each cert via the `renew()` closure in `certmagic@v0.21.3/config.go` (`... ensure ARI is updated ... if cert.NeedsRenewal(cfg) { RenewCertAsync/Sync } ... reloadManagedCertificate`), i.e. it checks **at config-load time** — matching Caddy author Matt Holt:

"*Caddy will immediately renew the certificate regardless of scan time, because it always checks when the config is first loaded.*" A continuous TLS probe (Q2) additionally drives the on-demand/ handshake maintenance path, so the async renewal completes promptly.

## Lever comparison (deployed stack: Pebble v2.6.0 + Caddy 2.8.4)

| Lever | Deterministic? | Zero-downtime? | Needs | Ver |
|---|---|---|---|---|
| **A. `renewal_window_ratio 1` + `POST /load` reload** | **YES** — ratio check returns true immediately, ARI cannot suppress it (source-proven) | **YES** — in-process renewal hot-swaps the cert; `POST /load` is a graceful reload | nothing new (uses the stubbed `CADDY_RENEWAL_RATIO` lever + admin API) | **RE** |
| B. Pebble `/set-renewal-info/` ARI override -> past window | YES (ARI-native) | YES (in-process) | **Pebble bump to >= v2.8.0** (endpoint added by PR #501, merged 2025-06-05; ABSENT in the deployed v2.6.0) + forcing an ARI re-fetch (ARI is cached ~`Retry-After 6h`) | Alte ope Peb mo |
| C. ACME-revoke the served leaf | Partial | YES (in-process) | out-of-band ACME revoke; Pebble returns `RenewalInfoImmediate` for revoked serials — but Caddy must re-fetch ARI (6h cache) to see it, and Pebble config disables OCSP (`ocspResponderURL:""`) so certmagic's OCSP-revoke `forceRenew` path is unavailable | Not sta |
| D. Delete cert from storage + **restart** Caddy | YES (fresh issuance, ARI-independent) | **NO** — process restart | nothing new | Goo SE "iss rota |

| Lever | Deterministic? | Zero-downtime? | Needs | Ver |
|---|---|---|---|---|
| | | drops connections | | **can** zer |

**Why not "delete cert from storage + *reload*" (no restart)?** Caddy issues #5589 and #6789 show a config *reload* does **not** reliably flush the in-memory cert cache / re-read storage for a same-identity cert — so delete+reload risks continuing to serve the cached leaf. That caveat is about **re-reading externally-modified/deleted files**, and is exactly why Lever A (which drives certmagic's *own* renewal codepath, replacing the cert in cache via `reloadManagedCertificate` on successful renewal) is preferred over any storage-surgery approach.

**Runtime-toggle wiring (important — the Caddyfile is mounted `:ro`).** `deploy/letsencrypt/Caddyfile` reads `renewal_window_ratio {$CADDY_RENEWAL_RATIO}` from process env, and env is fixed at container start; the Caddyfile mount is read-only. So the ratio cannot be flipped by editing the file in place. Two supported ways to apply ratio=1 at runtime without a restart:

- **(preferred) `POST /load` a config that carries a literal `renewal_window_ratio 1`.** Enable the admin API for the test (`CADDY_ADMIN=0.0.0.0:2019`, publish `127.0.0.1:2019:2019`), then `POST` the adapted config (Caddyfile body via `Content-Type: text/caddyfile`, or the JSON from `caddy adapt`) with the ratio line set to the literal `1`. This is a genuine config change -> full TLS reprovision -> `renew()` runs -> renew + hot-swap. Reset by `POST /load`-ing the original config afterward.
- **(simplest) boot with `CADDY_RENEWAL_RATIO=1` from the start.** Caddy then renews on every maintenance pass; the endpoint continuously rotates. The test samples the served leaf twice and asserts the serial changed with 0 dropped connections. Less controlled (no stable "before" baseline window) but requires no admin/config surgery. Revert to the default ratio when done.

Sources: Caddy `tls` directive + global options (`renewal_window_ratio`, default `0.3333`; ARI "is a suggestion … may not align with this ratio"); Caddy `API`/`automatic-https` (`POST /load` "incurs zero downtime", background cert management, renewal hot-swap); Matt Holt on force-renewal (ratio=1 + reload; delete+reload); Caddy issues #5589/#6789 (reload cert-cache caveat); certmagic `v0.21.3` `certificates.go`/`config.go`/`maintain.go` (read at tag). See footer.

# Q2 — Proving ZERO-DOWNTIME rotation: the concrete observable

**Primary evidence = the cert changed AND the endpoint never dropped.** Two independent captures taken across the forced renewal:

1. **Re-issue proof (the leaf actually rotated, not a cache reload).** Pull the *served* leaf before and after and compare two fields:

```
openssl s_client -connect 127.0.0.1:8443 -servername
proxy.hermetic.test </dev/null 2>/dev/null \
  | openssl x509 -noout -serial -startdate
# serial=<hex>     notBefore=<date>
```

PASS requires **`serial_after != serial_before`** AND **`notBefore_after > notBefore_before`**. A changed serial proves Pebble issued a NEW certificate (each Pebble issuance gets a fresh random serial); an advanced notBefore proves it was issued later (not the same cert re-served). Comparing only one field is insufficient — the serial alone could in principle collide; the pair is the rock-solid signal (§11.4.107 not-stale cross-check).

1. **Continuous-availability proof (no dropped connection across the swap).** Run a tight sampling loop against the TLS endpoint spanning the reload + the renewal swap, counting failures:

```
# sample every ~0.2s; a full TLS handshake + HTTP 200 each
iteration
while <rotation not yet observed>; do
  curl -sf --max-time 2 --resolve
proxy.hermetic.test:8443:127.0.0.1 \
       --cacert pebble_ca_bundle.pem \
       https://proxy.hermetic.test:8443/health >/dev/null \
    && ok=$((ok+1)) || fail=$((fail+1))
done
```

PASS requires **`fail == 0`** (every request completed a TLS handshake and got its response) across the entire window in which the serial transitions from `serial_before` to `serial_after`. This is the zero-downtime assertion: Caddy "swaps out the old certificate with the new one ... zero downtime" (automatic-https docs) and `POST /load` "incurs zero downtime" (API docs) — the probe MEASURES it rather than asserting it.

**Corroborating (not sole) evidence — Caddy logs.** With JSON logging, grep the certmagic/`tls` logger stream for the exact strings (verified in `certmagic@v0.21.3` source):

- `renewing certificate` (`config.go`) — renewal started;

- `certificate renewed successfully` (`config.go`) — renewal completed;
- `certificate is in configured renewal window based on expiration date` (`certificates.go`) — the ratio check fired (proves ratio=1 was the trigger, not ARI);
- (initial issuance, for the baseline: `obtaining certificate` -> `certificate obtained successfully`).

Treat the log as corroboration of *why/when*; the changed-serial + zero-failure pair is the proof. **UNCONFIRMED:** the exact wall-clock latency from `POST /load` to the new serial appearing (depends on Pebble VA scheduling + acmez retry) — the test polls the served serial with a bounded timeout rather than assuming a fixed delay; `PEBBLE_VA_NOSLEEP=1` + `PEBBLE_WFE_NONCEREJECT=0` (already set in the compose) keep it prompt.

**Do NOT conflate the reload blip with the renewal.** A *full config reload* (`POST /load`) reprovisions the TLS app; the *renewal* is the in-process hot-swap. Both are documented zero-downtime; the single continuous probe spans both, so `fail == 0` covers the whole operation. (This is also why the probe, not the log, is authoritative — issues #5589/#6789 show cache behavior around reload is subtle.)

Sources: Caddy automatic-https + API docs; certmagic `v0.21.3` log strings (read at tag). See footer.

---

# Q3 — Pebble-specific rotation quirks (deployed v2.6.0), source-confirmed

1. **Per-boot issuance-CA regeneration (unchanged from the hermetic doc — still load-bearing here).** Pebble regenerates its issuance root/intermediate on every launch and stores nothing on disk. **Consequence for Phase-5:** re-fetch the run's CA chain (`/roots/0` + `/intermediates/0`) at assertion time; NEVER pin a golden CA fixture. Because Phase-5 renews *within one Pebble boot*, `leaf_1` and `leaf_2` chain to the **same** run-CA — so the CA bundle is fetched once per test run and reused for both chain assertions (Q4). (If the stack is restarted mid-test, the CA changes and both leaves must be re-verified against the new bundle.)

2. **Pebble v2.6.0 DOES serve ARI — and its window is far in the future for a fresh cert.** Confirmed at tag `v2.6.0`: `wfe/wfe.go` exposes `renewalInfoPath = "/draft-ietf-acme-ari-03/renewalInfo/"`, advertised in the directory. `determineARIWindow(certID)`:
   - returns `core.RenewalInfoImmediate(now)` (a window **1 hour in the past**) **only if the serial is REVOKED**;
   - otherwise returns `core.RenewalInfoSimple(notBefore, notAfter)`, whose window is "*a point 2/3 of the way through the validity period, then a 2-day window around that*" (`core/types.go`: `idealRenewal = notAfter - validity/3`; window = `[idealRenewal - 24h, idealRenewal + 24h]`). With Pebble's default validity (see #3), a freshly-issued cert's ARI window starts **~3.3 years out**, so ARI on its own never triggers renewal in-test — and, per Q1, it also does NOT suppress the `renewal_window_ratio 1` trigger. **This is the fact that both (a) explains why time-based waiting is infeasible and (b) confirms Lever A is safe against ARI.** The `/set-renewal-info/` override endpoint that could move this window is **NOT in v2.6.0** (added by PR #501, merged 2025-06-05, first shipped in **v2.8.0**).

3. **Pebble default issued-leaf validity ~= 5 years, and Pebble HONORS a client-requested notBefore/notAfter (resolves a prior UNCONFIRMED).** `test/config/pebble-config.json@v2.6.0` sets `"certificateValidityPeriod": 157766400` seconds = **1826 days ~= 5 years**; `ca/ca.go@v2.6.0` computes `certNotAfter = certNotBefore + (validity-1)s`, **but** if the ACME order carries `notBefore`/`notAfter` it parses and honors them (`ca/ca.go@v2.6.0` lines ~272-286). So a client (Caddy's global `cert_lifetime`, which maps to the ACME order's notAfter) can request a SHORTER cert. **UNCONFIRMED:** whether `certmagic v0.21.3` actually populates the ACME order `notAfter` from `cert_lifetime` — verify at runtime with `openssl x509 -noout -startdate -enddate` on the issued leaf. Not needed for Lever A (validity-independent), but a short `cert_lifetime` is a useful secondary knob if a future test wants a time-based (rather than ratio-forced) renewal.

4. **ARI is client-cached ~6h (`Retry-After`), gated by `acmez NeedsRefresh()`.** `wfe/wfe.go@v2.6.0` sets `Retry-After: 21600` (6h) on the renewalInfo response; `certmagic@v0.21.3` only re-fetches ARI when `!DisableARI && cert.ari.NeedsRefresh()`. **Consequence:** any ARI-based lever (B or C) is not promptly deterministic without forcing an ARI re-fetch (delete stored ARI metadata or restart) — a second reason Lever A (which bypasses ARI via the unconditional ratio check) is the clean choice.

5. **Nonce/VA randomization (already mitigated in the deployed compose).** Pebble defaults: 0-15 s VA sleep, 5% nonce rejection, ~50% authz reuse. The compose already sets

`PEBBLE_VA_NOSLEEP=1` and `PEBBLE_WFE_NONCEREJECT=0` and runs `-strict false`; acmez retries nonces automatically. Keep per-attempt timeouts generous enough that a single retry does not surface as a failure in the availability probe.

6. **`Retry-After` on validation is separate** (`pebble-config.json@v2.6.0` `"retryAfter":{"authz":3,"order":5}`) — small, does not impede a single-boot renewal.

7. **The management interface is HTTPS on `:15000`, signed by `pebble.minica.pem`** — `curl -k` (or trust minica) to fetch `/roots/0` + `/intermediates/0`. The compose publishes it on `127.0.0.1:15000`, reachable by the conductor.

Sources: Pebble source at tag `v2.6.0` (`wfe/wfe.go`, `core/types.go`, `ca/ca.go`, `test/config/pebble-config.json`); Pebble Releases + PRs #461/#484/#501 (GitHub API); RFC 9773 / draft-ietf-acme-ari (ARI window semantics). See footer.

---

# Q4 — How `cert_analyzer.sh` plugs into the rotation assertion

The analyzer is client-and-challenge-agnostic and already provides every predicate the rotation test needs. Rotation adds only the **before/after** dimension (serial + notBefore delta), which is captured with `openssl` and asserted alongside the analyzer's PEM-level checks.

```
. tests/letsencrypt/cert_analyzer.sh

# --- run CA bundle (Pebble regenerates per boot; fetch ONCE per
test run, Q3.1) ---
curl -sk https://127.0.0.1:15000/roots/0         -o
pebble_root.pem
curl -sk https://127.0.0.1:15000/intermediates/0 -o
pebble_intermediate.pem
cat pebble_intermediate.pem pebble_root.pem      >
pebble_ca_bundle.pem   # bundle: Q4 note below

# --- BEFORE: served leaf #1 ---
openssl s_client -connect 127.0.0.1:8443 -servername
proxy.hermetic.test </dev/null 2>/dev/null \
  | openssl x509 > leaf_1.pem
serial_1=$(openssl x509 -in leaf_1.pem -noout -serial)
nb_1=$(openssl x509 -in leaf_1.pem -noout -startdate)

# --- (force renewal per Q1 Lever A; poll served leaf until serial
changes) -> leaf_2.pem ---
```

```
# --- AFTER: assert with the analyzer + the delta ---
cert_chain_roots_in leaf_2.pem pebble_ca_bundle.pem   # new cert
still ISSUED by the run's Pebble CA
cert_not_expired    leaf_2.pem                         # inside
validity now (honours CERT_ANALYZER_NOW_EPOCH seam)
cert_san_matches    leaf_2.pem proxy.hermetic.test     # SAN still
covers the served host
cert_days_remaining leaf_2.pem                          # scalar
for the evidence report
# rotation delta (NOT in the analyzer — captured with openssl):
#   serial_2 != serial_1   AND   nb_2 > nb_1
```

**Mapping (each analyzer function -> its role in the rotation PASS):**

| Assertion | Function / capture | PASS condition |
|---|---|---|
| A real re-issue happened | `openssl x509 -serial -startdate` before vs after | `serial_2 != serial_1` **and** `notBefore_2 > notBefore_1` |
| New cert issued by the run's Pebble CA | `cert_chain_roots_in leaf_2.pem pebble_ca_bundle.pem` | returns 0 |
| New cert valid now | `cert_not_expired leaf_2.pem [now_epoch]` | returns 0 |
| New cert covers the host | `cert_san_matches leaf_2.pem proxy.hermetic.test` | returns 0 |
| Zero-downtime | continuous TLS probe failure counter (Q2) | `fail == 0` |
| (report scalar) | `cert_days_remaining leaf_2.pem` | prints days |
| (regression seam, optional) | `cert_renewal_due leaf_1.pem <thresh> <now_epoch>` | using the `CERT_ANALYZER_NOW_EPOCH` seam, `leaf_1` can be *asserted* "due" at a synthetic now — the deterministic §11.4.115 RED baseline that a rotation then clears |

**Load-bearing integration note (matches the analyzer implementation).** `cert_chain_roots_in` runs `openssl verify -no_check_time -CAfile <expected_ca_pem> <leaf>`. Pebble issues a **leaf -> intermediate -> root** chain, so verifying against the **root alone** FAILS ("unable to get local issuer certificate"). Pass the **root+intermediate BUNDLE** as `expected_ca_pem` (as above) —

`openssl verify` treats every cert in `-CAfile` as a trust anchor, supplying the intermediate AND proving the leaf chains into the run's CA. `-no_check_time` keeps the chain check orthogonal to `cert_not_expired` (an expired-but-correctly-rooted cert still passes the chain check and fails the expiry check — by design, and useful for the §11.4.115 RED baseline). The `CERT_ANALYZER_NOW_EPOCH` seam (per-run) / trailing `now_epoch` arg (per-call) lets the same fixtures assert both "renewal was DUE before" and "the fresh leaf is valid now" with no time-travel (§11.4.50).

Sources: `tests/letsencrypt/cert_analyzer.sh` (in-repo); Pebble chain shape / "cannot get local issuer" (LE community + Pebble test/certs README). See footer.

---

# Phase-5 test PROCEDURE (conductor-executable against the already-built stack)

**Assumptions:** the hermetic stack is booted via the containers submodule (§11.4.76) from `deploy/letsencrypt/compose.hermetic.yml`; Caddy `localhost/helix_proxy/caddy-challtestsrv:2.8.4` has issued the Phase-3 baseline cert for `proxy.hermetic.test`; host ports per compose: Caddy HTTPS `127.0.0.1:8443`, Pebble mgmt `127.0.0.1:15000`. This procedure is **read/observe + a graceful reload only** — it does not stop/rebuild containers and does not touch `deploy/`, `wg0-mullvad`, `lava-*`, or `whoami:58080`.

**Determinism prerequisite (one-time, in the boot config):** enable the admin API for the test — set `CADDY_ADMIN=0.0.0.0:2019` and publish `127.0.0.1:2019:2019` (uncomment the admin port line in the compose). This is the only wiring the rotation test adds beyond Phase-3.

1. **Fetch the run's Pebble CA bundle (once).** `curl -sk https://127.0.0.1:15000/roots/0 -o pebble_root.pem; curl -sk https://127.0.0.1:15000/intermediates/0 -o pebble_intermediate.pem; cat pebble_intermediate.pem pebble_root.pem > pebble_ca_bundle.pem`.
2. **Baseline (leaf #1).** `openssl s_client -connect 127.0.0.1:8443 -servername proxy.hermetic.test </dev/null 2>/dev/null | openssl x509 > leaf_1.pem`; record `serial_1 = openssl x509 -in leaf_1.pem -noout -serial` and `nb_1 = ... -startdate`. Sanity: `cert_chain_roots_in leaf_1.pem pebble_ca_bundle.pem`, `cert_not_expired leaf_1.pem`, `cert_san_matches leaf_1.pem proxy.hermetic.test` all return 0.
3. **Start the availability probe (background).** Tight loop (~0.2 s) `curl -sf --max-time 2 --resolve proxy.hermetic.test:8443:127.0.0.1 --cacert`

`pebble_ca_bundle.pem https://proxy.hermetic.test:8443/health`, incrementing `ok/fail`. Keep it running through steps 4-5.

4. **Force renewal (Lever A).** `POST` to the admin API a config identical to the running one except the site's `renewal_window_ratio` is the literal `1`: `curl -X POST -H 'Content-Type: text/caddyfile' --data-binary @Caddyfile.ratio1 http://127.0.0.1:2019/load` (where `Caddyfile.ratio1` is the deployed Caddyfile with the `renewal_window_ratio {$CADDY_RENEWAL_RATIO}` line uncommented and set to literal `1`). The graceful reload re-manages the cert -> `certNeedsRenewal` returns true via the unconditional ratio check -> in-process renewal + hot-swap.
5. **Observe the rotation.** Poll `openssl s_client ... | openssl x509 -noout -serial -startdate` (bounded timeout, e.g. 60 s) until the serial differs from `serial_1`; capture `leaf_2.pem`, `serial_2`, `nb_2`. Corroborate in the Caddy JSON log: `renewing certificate` then `certificate renewed successfully` (+ `certificate is in configured renewal window based on expiration date`).
6. **Stop the probe; record `ok/fail`.**
7. **Assert (PASS requires ALL):**
   - `serial_2 != serial_1` **and** `nb_2 > nb_1` (real re-issue);
   - `cert_chain_roots_in leaf_2.pem pebble_ca_bundle.pem` == 0 (new cert roots in the run's Pebble CA);
   - `cert_not_expired leaf_2.pem` == 0 **and** `cert_san_matches leaf_2.pem proxy.hermetic.test` == 0;
   - `fail == 0` across the whole window (zero-downtime).
8. **Reset (rate-limit / renew-storm hygiene).** `POST /load` the ORIGINAL config (ratio back to the default `0.3333`) so Caddy stops renewing every maintenance pass. (Moot against Pebble, correct discipline for the staging/prod cutover.)
9. **Determinism (§11.4.50).** Repeat steps 2-8 N times (default 3), asserting identical PASS + identical assertion outcomes each iteration; because the renewal is within one boot, the CA bundle from step 1 is reused (Q3.1).
10. **Evidence (§11.4.69 / §11.4.107).** Persist under `qa-results/<run-id>/letsencrypt_rotation/`: `leaf_1.pem`, `leaf_2.pem`, the serial/notBefore captures, the Caddy renewal log lines, the availability `ok/fail` counters, and `pebble_ca_bundle.pem`.

**RED baseline (§11.4.115, optional but recommended).** Before the fix/lever is wired, the same harness with the availability probe but WITHOUT step 4 (or with a broken force-lever) must show the serial NEVER changes within the timeout -> the test FAILs (`serial_2 == serial_1`), proving the assertion genuinely catches "no rotation." Flipping in Lever A turns it GREEN.

**Anti-bluff (§11.4 / §11.4.107).** The proof is the tuple *(serial changed AND notBefore advanced AND new leaf chains to the run's Pebble CA AND new leaf valid AND 0 dropped connections)* with captured PEMs — never a log line alone, never

`PEBBLE_VA_ALWAYS_VALID` (which the compose correctly sets to `0`; setting it to `1` would not exercise DNS-01 and would be a §11.4 PASS-bluff for this phase).

---

# Honest gaps (UNCONFIRMED — runtime method stated inline; none block Phase-5)

- **Exact `POST /load` -> new-serial latency** (Pebble VA scheduling + acmez retries). Method: the procedure POLLS the served serial with a bounded timeout; `PEBBLE_VA_NOSLEEP=1` + `NONCEREJECT=0` (already set) keep it prompt.
- **Whether `certmagic v0.21.3` populates the ACME order `notAfter` from Caddy `cert_lifetime`** (Pebble v2.6.0 *would* honor it). Method: `openssl x509 -noout -startdate -enddate` on an issued leaf after setting `cert_lifetime`. Not needed for Lever A.
- **Whether the availability probe records literally 0 failures across the reload+swap on this host** — this is precisely what the test MEASURES (the zero-downtime claim is asserted from Caddy docs and PROVEN by the probe, not assumed).
- **Renewal sync vs async on the load path** (`config.go renew()` calls `RenewCertAsync`/`RenewCertSync` depending on context). The poll-until-serial-changes design is robust to either.

---

# Sources verified 2026-07-01

- Caddy `tls` directive (renewal_window_ratio, dns, resolvers, trusted_roots; ARI "is a suggestion") — https://caddyserver.com/docs/caddyfile/directives/tls
- Caddy global options (renewal_window_ratio default 0.3333; ARI/ratio note) — https://caddyserver.com/docs/caddyfile/options
- Caddy Automatic HTTPS (background cert management; renewal zero-downtime hot-swap) — https://caddyserver.com/docs/automatic-https
- Caddy API (`POST /load` "incurs zero downtime"; rollback on failure) — https://caddyserver.com/docs/api
- Caddy community — "How to force renewal of Let's Encrypt certificates" (Matt Holt: ratio=1 + reload "checks when the config is first loaded"; delete+reload) — https://caddy.community/t/how-to-force-renewal-of-lets-encrypt-certificates/14843
- Caddy issue #6789 — "reload --force won't recache certificates" (v2.9.0-beta.3, 2025-01-16; reload does not re-read storage for a same-identity cert) — https://github.com/caddyserver/caddy/issues/6789

- Caddy issue #5589 — cert cache flush on config reload — https://github.com/caddyserver/caddy/issues/5589
- Caddy issue #5516 — ARI support request — https://github.com/caddyserver/caddy/issues/5516
- certmagic `certificates.go` @v0.21.3 (`certNeedsRenewal`, `currentlyInRenewalWindow`) — https://github.com/caddyserver/certmagic/blob/v0.21.3/certificates.go
- certmagic `config.go` @v0.21.3 (load-time `renew()` path; ARI update; renew log strings) — https://github.com/caddyserver/certmagic/blob/v0.21.3/config.go
- certmagic `maintain.go` (master, RenewCheckInterval=10m default; ARI refresh gating; DisableARI) — https://github.com/caddyserver/certmagic/blob/master/maintain.go
- certmagic `certificates.go` (master, cross-check of the same logic + updated comments) — https://github.com/caddyserver/certmagic/blob/master/certificates.go
- Caddy `go.mod` @v2.8.4 (pins certmagic v0.21.3 + acmez/v2 v2.0.1) — https://github.com/caddyserver/caddy/blob/v2.8.4/go.mod
- certmagic ARI implementation PR #286 (Matt Holt) — https://github.com/caddyserver/certmagic/pull/286
- Pebble `wfe/wfe.go` @v2.6.0 (`renewalInfoPath`, `determineARIWindow`; no `/set-renewal-info/`) — https://github.com/letsencrypt/pebble/blob/v2.6.0/wfe/wfe.go
- Pebble `core/types.go` @v2.6.0 (`RenewalInfoSimple` 2/3-of-lifetime +/-1d window; `RenewalInfoImmediate` 1h-past window) — https://github.com/letsencrypt/pebble/blob/v2.6.0/core/types.go
- Pebble `ca/ca.go` @v2.6.0 (default validity + honors requested notBefore/notAfter) — https://github.com/letsencrypt/pebble/blob/v2.6.0/ca/ca.go
- Pebble `test/config/pebble-config.json` @v2.6.0 (`certificateValidityPeriod` 157766400s ~= 5y) — https://github.com/letsencrypt/pebble/blob/v2.6.0/test/config/pebble-config.json
- Pebble Releases (v2.6.0 2024-05-31, v2.7.0 2025-01-24, v2.8.0 2025-06-05, v2.9.0 2025-12-18, v2.10.1 2026-04-20) — https://github.com/letsencrypt/pebble/releases
- Pebble PR #461 "Implement latest draft-ietf-acme-ari spec" (merged 2024-05-24 -> in v2.6.0) — https://github.com/letsencrypt/pebble/pull/461
- Pebble PR #484 "return logical and compliant ARI windows for expiring certs" (merged 2025-02-21 -> v2.8.0) — https://github.com/letsencrypt/pebble/pull/484
- Pebble PR #501 "add overriding of ARI response" / `/set-renewal-info/` (merged 2025-06-05 -> v2.8.0) — https://github.com/letsencrypt/pebble/pull/501
- Pebble issue #403 "ACME Renewal Information (ARI)" (closed via #461) — https://github.com/letsencrypt/pebble/issues/403
- RFC 9773 — ACME Renewal Information (ARI) Extension (suggestedWindow start/end semantics) — https://datatracker.ietf.org/doc/rfc9773/

- Let's Encrypt — "An Engineer's Guide to Integrating ARI into Existing ACME Clients" (2024-04-25) — https://letsencrypt.org/2024/04/25/guide-to-integrating-ari-into-existing-acme-clients

**Source-file reads at pinned version tags (via `gh api`, 2026-07-01):** `letsencrypt/pebble@v2.6.0`: `wfe/wfe.go`, `core/types.go`, `ca/ca.go`, `test/config/pebble-config.json`. `caddyserver/caddy@v2.8.4`: `go.mod`. `caddyserver/certmagic@v0.21.3`: `certificates.go`, `config.go` (+ `maintain.go`/`certificates.go` @master for cross-check).